

OpenEMIS अलर्ट — प्रशासक मैनुअल

यह मैनुअल OpenEMIS Core अलर्ट मॉड्यूल के लिए आधिकारिक संदर्भ है, जिसे POCOR-9509 के तहत पेश किया गया और विस्तारित किया गया। यह प्रत्येक अलर्ट प्रकार, प्रत्येक कॉन्फ़िगरेशन विकल्प, और प्रत्येक परिचालन प्रक्रिया को कवर करता है जो एक प्रशासक को चाहिए। अपने कार्य के लिए प्रासंगिक अनुभाग पर सीधे नेविगेट करने के लिए विषय सूची का उपयोग करें। सामान्य समस्याओं के तेजी से समाधान के लिए पूरे दस्तावेज़ में कार्यशील उदाहरण और समस्या निवारण निर्णय वृक्ष प्रदान किए गए हैं।

विषय सूची

1. परिचय
 - 1.1 अलर्ट मॉड्यूल क्या करता है
 - 1.2 इस मैनुअल को कौन पढ़ना चाहिए
 - 1.3 POCOR-9509 में क्या नया है
 - 1.4 इस मैनुअल को कैसे पढ़ें
2. एक नजर में आर्किटेक्चर
 - 2.1 पाँच-चरण पाइपलाइन
 - 2.2 इवेंट-आधारित बनाम निर्धारित अलर्ट
 - 2.3 दो डिस्च पाथ
 - 2.4 शामिल डेटाबेस तालिकाएं
3. नेविगेशन और UI
 - 3.1 मॉड्यूल स्थान
 - 3.2 चार स्क्रीन
 - 3.3 अनुमतियाँ
4. अलर्ट शेड्यूल प्रबंधित करना
 - 4.1 अलर्ट प्रकारों की सूची
 - 4.2 आवृत्ति विकल्प
 - 4.3 अलर्ट शुरू और रोकना
 - 4.4 क्यों "कभी नहीं" सुरक्षित डिफ़ॉल्ट है
5. अलर्ट नियम — भेजने के लिए क्या कॉन्फ़िगर करें
 - 5.1 नियम शरीर रचना
 - 5.2 एक नियम बनाना
 - 5.3 एक नियम को संपादित और हटाना
 - 5.4 अलर्ट प्रकार के अनुसार कई नियम
 - 5.5 नियम शर्तें जो निष्पादन बंद करती हैं
6. प्लेसहोल्डर
 - 6.1 प्लेसहोल्डर सिंटैक्स
 - 6.2 सामान्य टोकन
 - 6.3 छात्र टोकन
 - 6.4 कर्मचारी और उपयोगकर्ता टोकन
 - 6.5 केस टोकन
 - 6.6 लाइसेंस टोकन
 - 6.7 छात्रवृत्ति टोकन
 - 6.8 सिस्टम अपडेट टोकन
 - 6.9 जब एक प्लेसहोल्डर शून्य या लापता हो तो व्यवहार
7. थ्रेसहोल्ड
 - 7.1 थ्रेसहोल्ड प्रारूप अवलोकन
 - 7.2 value फ़िल्ड
 - 7.3 condition फ़िल्ड
 - 7.4 वर्कफ़्लो-चरण थ्रेसहोल्ड
 - 7.5 श्रेणी और प्रकार फ़िल्टर
 - 7.6 युग्मित पूर्व/पश्च नियम बनाना
8. अलर्ट प्रकार संदर्भ
 - 8.1 छात्र अनुपस्थिति
 - 8.2 छात्र प्रवेश

- [8.3 छात्र नामांकन](#)
 - [8.4 छात्र स्थिति परिवर्तन](#)
 - [8.5 सेवानिवृत्ति चेतावनी](#)
 - [8.6 कर्मचारी रोजगार अंत](#)
 - [8.7 कर्मचारी छुट्टी अंत](#)
 - [8.8 कर्मचारी प्रकार](#)
 - [8.9 लाइसेंस वैधता](#)
 - [8.10 लाइसेंस नवीनीकरण](#)
 - [8.11 छात्रवृत्ति आवेदन](#)
 - [8.12 छात्रवृत्ति वितरण](#)
 - [8.13 केस एस्केलेशन](#)
 - [8.14 सिस्टम अपडेट](#)
 - [8.15 कर्मचारी उपस्थिति — कार्यान्वित नहीं](#)
9. [वर्कफ़्लो-ट्रिगर्ड अलर्ट](#)
- [9.1 वर्कफ़्लो अलर्ट नियम-आधारित अलर्ट से कैसे भिन्न हैं](#)
 - [9.2 वर्कफ़्लो अलर्ट को दबाने वाली शर्तें](#)
 - [9.3 वर्कफ़्लो अलर्ट सेट अप करना](#)
10. [अलर्ट क्यू — डिलीवरी पाइपलाइन](#)
- [10.1 क्यू स्क्रीन का उद्देश्य](#)
 - [10.2 क्यू कॉलम](#)
 - [10.3 स्थिति कोड](#)
 - [10.4 क्यू आइटम को बड़े पैमाने पर हटाना](#)
 - [10.5 क्यू जीवनचक्र आरेख](#)
11. [अलर्ट लॉग — ऑडिट ट्रेल](#)
- [11.1 अलर्ट लॉग का उद्देश्य](#)
 - [11.2 SHA-256 चेकसम के माध्यम से विस्थापन](#)
 - [11.3 एकल लॉग प्रविष्टियों को देखना और हटाना](#)
 - [11.4 लॉग को बड़े पैमाने पर हटाना](#)
12. [मेसेजिंग \(संस्था-स्तर\)](#)
- [12.1 संस्था-विशिष्ट Email और SMS सेटिंग्स](#)
 - [12.2 Default Messaging विकल्प](#)
 - [12.3 ईमेल और SMS कॉन्टैक्ट सत्यापन](#)
13. [परिचालन कॉन्फ़िगरेशन](#)
- [13.1 ALERTS PROCESS LIMIT और थ्रॉटलिंग](#)
 - [13.2 System Process शेड्यूल](#)
 - [13.3 अलर्ट लॉग को साफ़ करना](#)
14. [परीक्षण और सूखा चलाना प्रक्रिया](#)
- [14.1 अलर्ट नियम सूखा चलाना](#)
 - [14.2 व्यक्तिगत अलर्ट देखना](#)
 - [14.3 डेटाबेस अनुमति यात्रिकी को समझना](#)
 - [14.4 डेटाबेस अनुमति यात्रिकी को समझना](#)
 - [14.5 सभी निर्धारित अलर्ट को तुरंत चलाने को बाध्य करना](#)
15. [समस्या निवारण](#)
- [15.1 अलर्ट फायर किया गया लेकिन कोई ईमेल नहीं मिला](#)
 - [15.2 अलर्ट नियम सक्रम लेकिन कभी फायर नहीं होता](#)
 - [15.3 वर्कफ़्लो अलर्ट फायर नहीं हो रहा](#)
 - [15.4 क्यू में डुप्लिकेट अलर्ट](#)
 - [15.5 बड़े पैमाने पर हटाना सभी चयनित पंक्तियों को हटाता नहीं है](#)
 - [15.6 क्यू बैकिंग अप \(संदेश भेज नहीं रहे\)](#)
 - [15.7 कमांड लॉग फ़ाइलों को पढ़ना](#)
16. [परिशिष्ट](#)
- [A. पूर्ण Artisan कमांड संदर्भ](#)

- [B. तीन कमांड-मैप चेकलिस्ट](#)
- [C. SQL संदर्भ वचेरी](#)
- [D. शब्दावली](#)
- [E. आगे पढ़ना](#)

1. परिचय {#1-introduction}

1.1 अलर्ट मॉड्यूल क्या करता है {#11-what-the-alerts-module-does}

OpenEMIS अलर्ट मॉड्यूल एक ईवेंट-संचालित और निर्धारित सूचना प्रणाली है जो महत्वपूर्ण संस्था घटनाओं पर Email और SMS संदेश वितरित करती है। अलर्ट तब सक्रिय होते हैं जब:

- एक छात्र निर्दिष्ट दिनों में अनुपस्थित हो जाता है।
- कोई कर्मचारी सेवानिवृत्ति की तारीख के करीब आ जाता है।
- किसी कर्मचारी का शिक्षक प्रमाणपत्र समाप्त हो रहा है।
- एक कानूनी प्रक्रिया आगे बढ़ने के लिए प्रतीक्षा कर रही है।
- सिस्टम अद्यतन उपलब्ध है।

प्रत्येक अलर्ट प्रकार को परिभाषित **नियमों** के माध्यम से कॉन्फ़िगर किया जाता है, जो निर्दिष्ट करते हैं **कैसे** सूचित किया जाए, **क्या** कहना है, और **कितनी बार**। एक संस्था अधिकतर नियंत्रण के साथ प्रत्येक अलर्ट प्रकार के लिए कई नियम बना सकती है।

1.2 इस मैनुअल को कौन पढ़ना चाहिए {#12-who-should-read-this-manual}

यह मैनुअल निम्नलिखित के लिए लिखा गया है:

- OpenEMIS के लिए जिम्मेदार मंत्रालय IT प्रशासक।
- संचार और अधिसूचना की देखरेख करने वाले सिस्टम प्रबंधक।
- OpenEMIS का तैनाती इंजीनियर जो अलर्ट सिस्टम को कॉन्फ़िगर करता है।

यह मैनुअल **नहीं** है:

- OpenEMIS की स्थापना गाइड (अलर्ट मॉड्यूल तैनाती के लिए अलग दस्तावेज़ देखें)।
- डेवलपर का दस्तावेज़ (developers के लिए `ALERTS_GUIDE.md` देखें)।

1.3 POCOR-9509 में क्या नया है {#13-what-is-new-in-pocor-9509}

POCOR-9509 निम्नलिखित को पेश करता है:

- **अलर्ट क्यू स्क्रीन**। pending, sent, और failed messages का दृश्य।
- **बड़े पैमाने पर हटाना**। एक बटन में 100+ लॉग या क्यू आइटम को हटाएं।
- **पांच नई सतर्कता**। License Validity, License Renewal, Scholarship Application, Scholarship Disbursement, और Case Escalation।
- **Laravel-आधारित कमांड संचलन**। CakePHP `afterSave` hooks से Laravel Artisan कमांड में स्विच करें।
- **थ्रॉटलिंग**। `ALERTS_PROCESS_LIMIT` environment variable के माध्यम से एक समय में कितने messages भेजें।

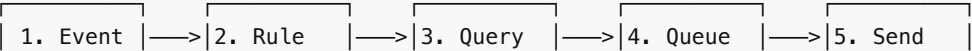
1.4 इस मैनुअल को कैसे पढ़ें {#14-how-to-read-this-manual}

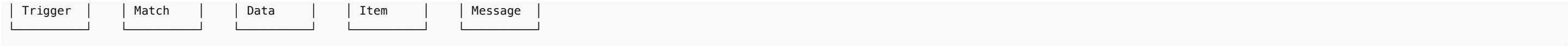
यह एक **संदर्भ मैनुअल** है। शुरू से अंत तक पढ़ने की जरूरत नहीं है।

- अगर आप **नई शुरुआत कर रहे हैं**, खंड 2 (Architecture) पढ़ें, फिर खंड 3 (Navigation) और खंड 4 (Managing Schedules)।
- अगर आप **एक अलर्ट नियम जोड़ रहे हैं**, खंड 5 (Alert Rules) और खंड 8 (Alert Types) के लिए सीधे जाएं।
- अगर आप **समस्या का सामना कर रहे हैं**, खंड 15 (Troubleshooting) को देखें।
- सभी **प्लेसहोल्डर टोकन** के लिए, खंड 6 (Placeholders) को संदर्भित करें।

2. एक नजर में आर्किटेक्चर {#2-architecture-at-a-glance}

2.1 पाँच-चरण पाइपलाइन {#21-the-five-stage-pipeline}





Stage 1 — Event Trigger: एक परिवर्तन (छात्र जोड़ा गया, कर्मचारी छुट्टी अंत) CakePHP `afterSave` hook को ट्रिगर करता है या निर्धारित `alerts:check-and-queue` कमांड चलता है।

Stage 2 — Rule Match: सिस्टम सभी सक्षम नियमों की जांच करता है जो इस ईवेंट को सुनते हैं। Threshold मान और शर्तें जाँच की जाती हैं। नियम disabled हैं, या threshold मान्य नहीं है, या कोई सुरक्षा भूमिका नहीं है → **कोई अलर्ट नहीं**।

Stage 3 — Query Data: यदि नियम मेल खाता है, सिस्टम database से तथ्य इकट्ठा करता है: संस्था नाम, छात्र विवरण, शिक्षक नाम, आदि।

Stage 4 — Queue Item: सिस्टम `alert_queue` में एक या अधिक rows डालता है, प्रत्येक recipient (ईमेल पते या फोन नंबर) के लिए एक। प्रत्येक item में status = "pending" होता है।

Stage 5 — Send Message: एक background cron process (Laravel का `alerts:process` कमांड) queue से items को खींचता है और Email (SMTP के माध्यम से) या SMS (Twilio के माध्यम से) भेजता है। Success → status = "sent", Failure → status = "failed"।

2.2 ईवेंट-आधारित बनाम निर्धारित अलर्ट {#22-event-based-vs-scheduled-alerts}

पहलू	ईवेंट-आधारित	निर्धारित
कब ट्रिगर होता है	जब कोई परिवर्तन होता है (छात्र जोड़ा, कर्मचारी अद्यतन)	दैनिक, साप्ताहिक, या कभी नहीं (frequency फ़ील्ड के आधार पर)
कमांड कहाँ चलता है	CakePHP afterSave hook से तुरंत	निर्धारित <code>alerts:check-and-queue</code> (cron) से
उदाहरण अलर्ट	StudentAbsence, StudentAdmission, StaffLeave	LicenseValidity, CaseEscalation, Scholarship Disbursement
Start/Stop बटन	कोई नहीं (हमेशा सक्षम)	Alerts screen पर Yes (frequency को "Never" सेट करने के लिए)

2.3 दो डिस्पैच पाथ {#23-the-two-dispatch-paths}

अलर्ट दो रूटों से Laravel Artisan कमांड तक पहुंचते हैं:

Path	कहाँ से	कैसे
Event-based	CakePHP plugins/Alert/src/Model/Table/AlertLogsTable.php में afterSave hook	<code>AlertLogsTable::triggerAlertCommand()</code> तुरंत Laravel में काम करता है
Scheduled	कमांड लाइन से cron में <code>alerts:check-and-queue</code>	<code>CheckAndQueueAlerts::queueAlertCommand()</code> सभी pending scheduled alerts को खोजता है

महत्वपूर्ण: एक alert को **दोनों** maps में सूचीबद्ध नहीं किया जा सकता; यह डुप्लिकेट deliver करेगा।

2.4 शामिल डेटाबेस तालिकाएं {#24-database-tables-involved}

तालिका	उद्देश्य
alerts	प्रत्येक alert प्रकार की पंक्ति (StudentAbsence, CaseEscalation, आदि)। frequency फ़ील्ड alert को सक्षम/अक्षम करता है।
alert_rules	प्रशासक द्वारा बनाए गए नियम। Fields: name, enabled, feature, method, threshold, recipient specification।
alert_queue	भेजने के लिए pending messages (status = pending/sent/failed)। Queue lifecycle को track करता है।
alert_logs	सभी भेजे गए messages का audit trail। Deduplication के लिए SHA-256 checksums शामिल।
alert_logs_roles	audit trail के लिए किस security role को सूचित किया गया।
security_groups	recipients के नाम के समूह (e.g., "School Principals")।
security_users	individual users (email/phone contact info के साथ)।
system_processes	कमांड execution logs। Each <code>alerts:*</code> कमांड एक row यहाँ बनाता है।

3. नेविगेशन और UI {#3-navigation-and-ui}

3.1 मॉड्यूल स्थान {#31-module-location}

अलर्ट मॉड्यूल यहाँ स्थित है:

Administration → Communications → Alerts

इसमें चार sub-screens हैं:

3.2 चार स्क्रीन {#32-the-four-screens}

स्क्रीन	पाथ	उद्देश्य
Alerts	Administration > Communications > Alerts	प्रत्येक alert प्रकार की frequency को enable/disable/configure करें।
Alert Rules	Administration > Communications > Alert Rules	नियम बनाएं, संपादित करें, हटाएं जो निर्दिष्ट करते हैं कि किसे और क्या भेजा जाए।
Alert Queue	Administration > Communications > Alert Queue	Pending और failed messages को देखें। Mass delete।
Alert Logs	Administration > Communications > Alert Logs	सभी भेजे गए messages का audit trail। Deduplication checksums।

3.3 अनुमतियाँ {#33-permissions}

तीन अनुमति स्तर हैं:

अनुमति	क्या यह करने देता है
View	सभी चार स्क्रीन को देखता है, लेकिन संपादन या हटाने में सक्षम नहीं है।
Execute	Alert rules बना, संपादित, हटा सकता है। Alert frequency बदल सकता है।
Full	सभी के ऊपर, साथ ही queue और logs से आइटम को mass delete कर सकता है।

अनुमतियों को configure करने के लिए:

Security → Roles → [role name] → Communications → Alerts

4. अलर्ट शेड्यूल प्रबंधित करना {#4-managing-alert-schedules}

4.1 अलर्ट प्रकारों की सूची {#41-the-alert-types-list}

Alerts screen सभी 15 alert प्रकारों को सूचीबद्ध करता है। प्रत्येक row में है:

- **Name:** alert का नाम (e.g., "Student Absence")।
- **Code:** system identifier (e.g., StudentAbsence)।
- **Frequency:** Never (disabled), Once, Daily, Weekly, Monthly।
- **Action buttons:** Edit, Delete (if no rules depend)।

4.2 आवृत्ति विकल्प {#42-frequency-options}

आवृत्ति	मतलब
Never	यह alert type fully disabled है। नियम तब भी exist हो सकते हैं, लेकिन कभी execute नहीं होते। यह सुरक्षित डिफ़ॉल्ट है।
Once	Alert check करें, फिर स्वचालित रूप से "Never" पर बदलें। (Test के लिए उपयोगी)।
Daily	हर 24 घंटे में check करें। SystemUpdates के लिए default।
Weekly	हर 7 दिनों में check करें।
Monthly	हर 30 दिनों में check करें।

4.3 अलर्ट शुरू और रोकना {#43-starting-and-stopping-an-alert}

एक scheduled alert को शुरू करने के लिए (frequency = "Never" से कुछ और तक बदलें):

1. **Alerts** screen पर जाएं।
2. alert row में **Edit** बटन पर क्लिक करें।
3. **Frequency** dropdown को "Never" से "Daily" (या अन्य विकल्प) में बदलें।
4. **Save** बटन क्लिक करें।

एक scheduled alert को रोकने के लिए:

1. **Alerts** screen पर जाएं।
2. alert row में **Edit** बटन पर क्लिक करें।
3. **Frequency** को "Never" पर सेट करें।
4. **Save** बटन क्लिक करें।

Note: Event-based alerts (जैसे StudentAbsence) का Start/Stop बटन नहीं है। वे हमेशा सक्रम हैं जब कम से कम एक नियम enabled है।

4.4 क्यों "कभी नहीं" सुरक्षित डिफ़ॉल्ट है {#44-why-never-is-the-safe-default}

Frequency = "Never" मतलब:

- नियम तब भी exist हो सकते हैं और visible हो सकते हैं।
- लेकिन cron कभी execute नहीं होगा।
- Email/SMS तब तक भेजे नहीं जाएंगे जब तक आप explicitly frequency को सक्रम न करें।

यह unintended mass notifications से बचाता है जब आप alert नियमों को test कर रहे हों।

5. अलर्ट नियम — भेजने के लिए क्या कॉन्फ़िगर करें {#5-alert-rules--configuring-what-to-send}

5.1 नियम शरीर रचना {#51-rule-anatomy}

एक alert rule में ये fields होते हैं:

फ़ील्ड	विवरण	उदाहरण
Name	नियम के लिए एक descriptive label ।	"12-दिन का अनुपस्थिति चेतावनी"
Enabled	यह नियम चलना चाहिए या नहीं।	Yes / No
Feature	किस alert type के लिए यह नियम है।	StudentAbsence, CaseEscalation, आदि।
Method	कैसे सूचित करें: "email" या "sms" ।	Email
Threshold	कौन सी conditions match करना चाहिए।	JSON: {"value": 12}
Recipients	कौन सी security roles notify हों।	Classroom Teachers, Deputy Principals
Subject	Email/SMS subject template ।	"अनुपस्थिति चेतावनी: \${student.name}"
Message	Email/SMS body template ।	"Student \${student.name} has been absent for \${days} days."

5.2 एक नियम बनाना {#52-creating-a-rule}

1. **Alert Rules** screen पर जाएं।
2. **Add** ("+") बटन क्लिक करें।
3. निम्नलिखित को भरें:
 - **Rule Name:** कुछ descriptive, जैसे "License Expiring in 30 Days" ।
 - **Alert Type (Feature):** dropdown से चुनें।
 - **Enabled:** Yes सेट करें।
 - **Method:** Email या SMS चुनें।

- **Threshold JSON:** JSON object दर्ज करें (section 7 में प्रारूप देखें)।
- **Notify Roles:** checkbox के साथ roles चुनें।
- **Subject Template:** अपना subject text दर्ज करें (placeholders के साथ)।
- **Message Template:** अपना message text दर्ज करें।

4. **Save** बटन क्लिक करें।

5.3 एक नियम को संपादित और हटाना {#53-editing-and-deleting-a-rule}

संपादित करना:

1. **Alert Rules** screen पर जाएं।
2. rule row में **Edit** बटन क्लिक करें।
3. अपने changes बनाएं।
4. **Save** क्लिक करें।

हटाना:

1. **Alert Rules** screen पर जाएं।
2. rule row में **Delete** (trash icon) बटन क्लिक करें।
3. confirmation dialog में **Yes** क्लिक करें।

5.4 अलर्ट प्रकार के अनुसार कई नियम {#54-multiple-rules-per-alert-type}

यह अलर्ट सिस्टम का सबसे शक्तिशाली feature है। एक single alert type (जैसे LicenseValidity) के लिए आप **cascading rules** का निर्माण कर सकते हैं:

उदाहरण 1: License Validity Tiered Warnings

निम्नलिखित तीन नियम बनाएं:

1. **"License expiring in 60 days"**
 - Threshold: {"value": 60, "license_type": 3, "condition": 1}
 - Recipients: HR Managers
 - Subject: "🔴 License Renewal Required — 60 Days Left"
2. **"License expiring in 30 days"**
 - Threshold: {"value": 30, "license_type": 3, "condition": 1}
 - Recipients: HR Managers, Deputy Principals
 - Subject: "🔴🟡 License Renewal Urgent — 30 Days Left"
3. **"License already expired"**
 - Threshold: {"value": 7, "license_type": 3, "condition": 2}
 - Recipients: HR Managers, Principal, HR Officer
 - Subject: "🔴🟡🟡 License Expired — Immediate Action Required"

जब alerts:license-validity cron runs, यह सभी तीनों नियमों को जांचता है। एक कर्मचारी जिसका license 25 दिनों में समाप्त है, rules 1 और 2 दोनों को match करेगा, और **दोनों** को notify receive करेंगे।

उदाहरण 2: Case Escalation Multi-Step Workflow

```
Rule 1: Escalation after 3 days
Threshold: {"workflow_steps": [5], "value": 3, "condition": 1}
Recipients: Case Managers
Subject: "Case escalation pending – 3 days"

Rule 2: Escalation after 7 days
Threshold: {"workflow_steps": [5], "value": 7, "condition": 1}
Recipients: Case Managers, Supervisors
Subject: "Case escalation overdue – 7 days"
```

```
Rule 3: Escalation after 21 days
  Threshold: {"workflow_steps": [5], "value": 21, "condition": 1}
  Recipients: Case Managers, Supervisors, Director
  Subject: "CRITICAL: Case escalation 3 weeks overdue"
```

यह patterns allow करते हैं जहाँ escalation notifications differently target करते हैं जैसे ही case older हो जाता है।

5.5 नियम शर्तें जो निष्पादन बंद करती हैं {#55-rule-conditions-that-stop-execution}

एक नियम निष्पादित नहीं होगा अगर:

शर्त	परिणाम
Enabled = No	नियम skip किया जाता है।
Alert type frequency = Never	पूरा alert type disabled है।
Threshold JSON invalid	सिस्टम error log करता है और niyam को skip करता है।
Recipient roles empty	कोई recipients नहीं है, तो कोई messages queue में नहीं जाते।
Data does not match threshold	Example: "60-day warning" threshold but staff license expires in 45 days → no match.

6. प्लेसहोल्डर {#6-placeholders}

6.1 प्लेसहोल्डर सिंटैक्स {#61-placeholder-syntax}

Placeholder tokens एक `${...}` format में messages में डाले जाते हैं। ये **case-sensitive** हैं।

Template उदाहरण:

```
Dear ${institution.name},

Student ${student.name} (OpenEMIS ID: ${student.openemis_no})
has been absent for ${days_absent} days.

Regards,
OpenEMIS
```

Resolved उदाहरण:

```
Dear Government School No. 5,

Student Muhammad Hassan (OpenEMIS ID: 0E000001234)
has been absent for 12 days.

Regards,
OpenEMIS
```

6.2 सामान्य टोकन {#62-common-tokens}

ये सभी alert types में available हैं:

टोकन	मान
<code>\${institution.code}</code>	संस्था की ministry code (जैसे "GS005")।
<code>\${institution.name}</code>	संस्था का full name (जैसे "Government School No. 5")।

\${institution.city}	संस्था का city।
\${institution.district}	संस्था का district।

6.3 छात्र टोकन {#63-student-tokens}

Student-related alerts में use करें (StudentAbsence, StudentAdmission, StudentEnrolment, StudentStatus):

टोकन	मान
\${student.name}	Student का full name।
\${student.openemis_no}	OpenEMIS student identifier (जैसे "OE123456")।
\${student.date_of_birth}	Date of birth (format: YYYY-MM-DD)।
\${student.gender}	M, F, or Other।
\${student.address}	Residential address।

6.4 कर्मचारी और उपयोगकर्ता टोकन {#64-staff-and-user-tokens}

Staff-related alerts में (RetirementWarning, StaffEmployment, StaffLeave, StaffType, LicenseValidity, LicenseRenewal):

टोकन	मान
\${staff.name}	Staff member का full name।
\${staff.openemis_no}	OpenEMIS staff identifier।
\${staff.email}	Staff का email address।
\${staff.date_of_birth}	Date of birth।
\${staff.start_date}	Employment start date।
\${staff.retirement_date}	Expected retirement date (RetirementWarning में)।

6.5 केस टोकन {#65-case-tokens}

CaseEscalation alert में:

टोकन	मान
\${case.id}	Case ID।
\${case.name}	Case name/title।
\${case.description}	Case description।
\${case.assigned_to}	Assigned person का name।
\${case.current_step}	Current workflow step का name।
\${case.days_in_step}	कितने दिन current step में है।

6.6 लाइसेंस टोकन {#66-license-tokens}

LicenseValidity, LicenseRenewal में:

टोकन	मान
\${license.type}	License का type (जैसे "Elementary Education") ।
\${license.issued_date}	Issue date ।
\${license.expiry_date}	Expiry date ।
\${license.days_remaining}	Expiry तक बचे दिन (negative अगर expired) ।

6.7 छात्रवृत्ति टोकन {#67-scholarship-tokens}

ScholarshipApplication, ScholarshipDisbursement में:

टोकन	मान
\${scholarship.name}	Scholarship program का नाम।
\${scholarship.amount}	Award amount ।
\${student.name}	Recipient student का name ।
\${application.id}	Application ID ।
\${application.status}	Current status ।

6.8 सिस्टम अपडेट टोकन {#68-system-update-tokens}

SystemUpdates alert में:

टोकन	मान
\${new_version}	Available version number (जैसे "v5.2.1") ।
\${release_date}	Release date ।
\${current_version}	Currently deployed version ।

6.9 जब एक प्लेसहोल्डर शून्य या लापता हो तो व्यवहार {#69-behaviour-when-a-placeholder-is-null-or-missing}

अगर एक placeholder का value null है या unknown है:

स्थिति	Result
Placeholder में value है	Value को message में डाला जाता है।
Placeholder में no value है	String को समान रूप से जैसे दिखता है दिखाया जाता है, जैसे \${staff.email} → \${staff.email} ।
Placeholder typo है	जैसे \${stafff.name} → \${stafff.name} (unchanged) ।
Optional token अनुपलब्ध है	Message delivery continue होता है; recipient को partial message मिलता है।

7. थ्रेशहोल्ड {#7-thresholds}

7.1 थ्रेशहोल्ड प्रारूप अवलोकन {#71-threshold-formats-overview}

Threshold JSON object है जो निर्दिष्ट करता है कि कौन सी conditions alert को trigger करें। Format alert type के आधार पर भिन्न होता है।

Alert Type	Format	उदाहरण
------------	--------	--------

StudentAbsence	Integer (days)	30 = 30 दिनों की अनुपस्थिति
LicenseValidity	JSON object	{"value": 60, "license_type": 3, "condition": 1}
CaseEscalation	JSON with array	{"workflow_steps": [5, 7], "value": 21, "condition": 1}
StaffType	JSON with filter	{"staff_type_id": 2, "value": 0}
ScholarshipApplication	JSON with category	{"category": "merit", "value": 7}

7.2 value फ़िल्ड {#72-the-value-field}

value field ज्यादातर cases में **days** है:

```
{
  "value": 30,
  "license_type": 3,
  "condition": 1
}
```

इसका मतलब: "Alert fire करें अगर license expiry की तारीख **30 days से कम दूर है**" (condition=1 के साथ)।

value के विभिन्न interpretations alert type पर depend करते हैं:

Alert Type	value meaning
StudentAbsence	Days absent (threshold: छात्र कम से कम इतने दिन absent हो चुका है)।
LicenseValidity	Days before expiry (condition=1) or days after expiry (condition=2)।
CaseEscalation	Days in current workflow step।
StaffType	Not applicable (value=0)।

7.3 condition फ़िल्ड {#73-the-condition-field}

condition field निर्दिष्ट करता है कि value से **पहले** या **बाद में** trigger करें:

Condition	Meaning
1	"Before" — value की तुलना में कम या बराबर । Example: expiry से 60 days पहले trigger करें।
2	"After" — value की तुलना में बराबर या ज्यादा । Example: expire होने के 7 days बाद trigger करें।

SQL Semantics:

```
-- condition = 1 (BEFORE)
WHERE DATEDIFF(expiry_date, CURDATE()) <= value

-- condition = 2 (AFTER)
WHERE DATEDIFF(CURDATE(), expiry_date) >= value
```

7.4 वर्कफ़्लो-चरण थ्रेशहोल्ड {#74-workflow-step-thresholds}

CaseEscalation जैसे alerts के लिए, आप specify कर सकते हैं कि कौन सी **workflow steps** alert को trigger करें:

```
{
  "workflow_steps": [5, 7, 10],
  "value": 14,
  "condition": 1
}
```

इसका मतलब: "Alert करें अगर case step 5, 7, या 10 में है और उस step में 14 days से ज्यादा रहा है।"

Workflow step IDs खोजने के लिए:

```
SELECT ws.id, ws.name, w.name AS workflow
FROM workflow_steps ws
JOIN workflows w ON w.id = ws.workflow_id
ORDER BY w.name, ws.name;
```

7.5 श्रेणी और प्रकार फ़िल्टर {#75-category-and-type-filters}

कुछ alert types allow करते हैं filtering by category या type:

Alert Type	Filter Field	Meaning	Example
StaffLeave	staff_leave_type	Only trigger for specific leave types।	{"staff_leave_type": 3} = Sick Leave
StaffType	staff_type_id	Only trigger for specific staff types।	{"staff_type_id": 2} = Teachers
LicenseRenewal	staff_training_categories	Only trigger for specific training categories।	{"staff_training_categories": [1, 3]}
LicenseValidity	license_type	Only trigger for specific license types।	{"license_type": 5} = Secondary License

Available IDs खोजने के लिए SQL:

```
-- Staff leave types
SELECT id, name FROM staff_leave_types ORDER BY name;

-- Staff types
SELECT id, name FROM staff_types ORDER BY name;

-- License types
SELECT id, name FROM license_types ORDER BY name;

-- Training categories
SELECT DISTINCT category FROM staff_training_categories ORDER BY category;
```

7.6 युग्मित पूर्व/पश्च नियम बनाना {#76-creating-paired-beforeafter-rules}

जब आप "before" और "after" दोनों notifications चाहते हैं, दो नियम बनाएं:

Rule 1: Before expiry

```
{
  "value": 30,
  "condition": 1
}
```

Subject: "License expiring in 30 days"

Rule 2: After expiry

```
{
  "value": 7,
  "condition": 2
}
```

Subject: "License expired 7 days ago"

दोनों नियमों को same alert type (जैसे LicenseValidity) में अलग-अलग recipients के साथ सेटअप करें।

8. अलर्ट प्रकार संदर्भ {#8-alert-types-reference}

8.1 छात्र अनुपस्थिति {#81-student-absence}

Trigger: Event-based (जब attendance record create/update होता है)

Threshold Format: Integer (days)

उदाहरण:

```
30
```

(30 दिन से ज्यादा absent छात्र को trigger करें)

Worked Example:

Rule Name: "Student absent for 12 days"

- Threshold: 12
- Recipients: Classroom Teachers, Deputy Principals
- Subject: Attendance Alert: \${student.name}
- Message: Student \${student.name} (\${student.openemis_no}) has been absent for \${days_absent} days.

8.2 छात्र प्रवेश {#82-student-admission}

Trigger: Event-based (जब student admission create होता है)

Threshold Format: None (JSON not applicable)

Worked Example:

Rule Name: "New admission notification"

- Recipients: Principal, Vice Principal
- Subject: New Student Admission: \${student.name}
- Message: A new student, \${student.name}, has been admitted to class \${student.class_name}.

8.3 छात्र नामांकन {#83-student-enrolment}

Trigger: Event-based (जब student enrol होता है)

Threshold Format: None

Worked Example:

Rule Name: "Student enrollment notification"

- Recipients: Registrar
- Subject: Student Enrolled: \${student.name}

- Message: `${student.name}` has been enrolled in class `${student.class_name}` for academic year `${academic_year}`.

8.4 छात्र स्थिति परिवर्तन {#84-student-status-change}

Trigger: Event-based (जब student status change होता है)

Threshold Format: JSON (status filter)

उदाहरण:

```
{
  "student_status_id": 5
}
```

(केवल जब status = 5 हो [e.g., Dropout])

Worked Example:

Rule Name: "Student dropout alert"

- Threshold: `{"student_status_id": 5}`
- Recipients: Principal, Academic Coordinator
- Subject: `ALERT: Student Dropped Out – ${student.name}`
- Message: `Student ${student.name} has changed status to Dropout. Immediate follow-up recommended.`

8.5 सेवानिवृत्ति चेतावनी {#85-retirement-warning}

Trigger: Scheduled (daily)

Threshold Format: Integer (days before retirement)

उदाहरण:

```
365
```

(सेवानिवृत्ति से 1 साल पहले alert करें)

Worked Example:

Rule Name: "Staff retirement in 6 months"

- Threshold: `180` (6 महीने पहले)
- Recipients: HR Manager, Principal
- Subject: `Staff Retirement Notice: ${staff.name}`
- Message: `${staff.name} will retire on ${staff.retirement_date}. Please plan for replacement.`

8.6 कर्मचारी रोजगार अंत {#86-staff-employment-end}

Trigger: Event-based (जब employment record end होता है)

Threshold Format: None

Worked Example:

Rule Name: "Staff employment ended"

- Recipients: HR Manager
- Subject: `Employment Ended: ${staff.name}`
- Message: `${staff.name}'s employment at ${institution.name} has ended as of ${employment_end_date}.`

8.7 कर्मचारी छुट्टी अंत {#87-staff-leave-end}

Trigger: Event-based (जब staff leave record end होता है)

Threshold Format: JSON (leave type filter)

उदाहरण:

```
{
  "staff_leave_type": 3
}
```

(केवल Sick Leave के लिए)

Worked Example:

Rule Name: "Staff returning from leave"

- Threshold: {"staff_leave_type": 1} (Annual Leave)
- Recipients: Principal, HR Manager
- Subject: Staff Resuming Duty: \${staff.name}
- Message: \${staff.name} is resuming duty after \${leave_type} leave. Expected return date: \${return_date}.

8.8 कर्मचारी प्रकार {#88-staff-type}

Trigger: Scheduled (daily)

Threshold Format: JSON (staff type filter)

उदाहरण:

```
{
  "staff_type_id": 2,
  "value": 0
}
```

(Teachers केवल)

Worked Example:

Rule Name: "Report on all teachers"

- Recipients: HR Manager
- Subject: Daily Teacher Report – \${institution.name}
- Message: There are \${total_teachers} teachers at \${institution.name}.

8.9 लाइसेंस वैधता {#89-license-validity}

Trigger: Scheduled (daily)

Threshold Format: JSON (days, license type, condition)

उदाहरण:

```
{
  "value": 60,
  "license_type": 3,
  "condition": 1
}
```

(License type 3 [Elementary] expiring in 60 days)

Worked Example — Tiered warnings:

Rule 1: 90 days before expiry

- Threshold: {"value": 90, "license_type": 3, "condition": 1}
- Recipients: HR Manager
- Subject: License Renewal Notice – 90 Days – \${staff.name}

Rule 2: 30 days before expiry

- Threshold: {"value": 30, "license_type": 3, "condition": 1}
- Recipients: HR Manager, Principal
- Subject: License Renewal Urgent – 30 Days – \${staff.name}

Rule 3: Already expired

- Threshold: {"value": 7, "license_type": 3, "condition": 2}
- Recipients: HR Manager, Principal, Director
- Subject: CRITICAL: License Expired – \${staff.name}

8.10 लाइसेंस नवीनीकरण {#810-license-renewal}

Trigger: Scheduled (daily)

Threshold Format: JSON (value, license_type, condition)

समान 8.9 को लेकिन renewal-specific logic के साथ।

8.11 छात्रवृत्ति आवेदन {#811-scholarship-application}

Trigger: Scheduled (daily)

Threshold Format: JSON (category, value)

महत्वपूर्ण: Recipient **assignee केवल** है, role-based नहीं। यदि कोई application का assignee नहीं है, कोई alert नहीं भेजा जाता।

उदाहरण:

```
{
  "category": "merit",
  "value": 7
}
```

(Merit-based applications 7+ दिनों के pending)

Worked Example:

Rule Name: "Scholarship applications pending 7+ days"

- Threshold: {"category": "merit", "value": 7}
- Subject: Scholarship Application Status: \${scholarship.name}
- Message: Application ID \${application.id} for \${student.name} has been pending for 7 days. Your attention is required.

8.12 छात्रवृत्ति वितरण {#812-scholarship-disbursement}

Trigger: Scheduled (daily)

Threshold Format: JSON (amount, value)

महत्वपूर्ण: Recipients **role-based** हैं (institution-agnostic)।

उदाहरण:


```
{
  "amount": 500,
  "value": 0
}
```

(सभी disbursements)

Worked Example:

Rule Name: "Scholarship disbursement notification"

- Recipients: Finance Officer, Headmaster
- Subject: Scholarship Disbursement: \${scholarship.name}
- Message: Scholarship amount \${scholarship.amount} has been disbursed to \${student.name}.

8.13 केस एस्केलेशन {#813-case-escalation}

Trigger: Scheduled (daily)

Threshold Format: JSON (workflow_steps array, value, condition)

महत्वपूर्ण: Recipients = assignee union roles। Assignee को हमेशा notify किया जाता है, साथ ही any configured roles।

उदाहरण:

```
{
  "workflow_steps": [5, 7],
  "value": 14,
  "condition": 1
}
```

(Steps 5 या 7 में, 14 दिनों से ज्यादा)

Worked Example — Multi-escalation:

Rule 1: 3-day escalation

- Threshold: {"workflow_steps": [5], "value": 3, "condition": 1}
- Recipients: Case Manager
- Subject: Case Escalation: \${case.name} (3 Days)

Rule 2: 7-day escalation

- Threshold: {"workflow_steps": [5], "value": 7, "condition": 1}
- Recipients: Case Manager, Supervisor
- Subject: Case Escalation: \${case.name} (7 Days) – Urgent

Rule 3: 21-day escalation

- Threshold: {"workflow_steps": [5], "value": 21, "condition": 1}
- Recipients: Case Manager, Supervisor, Director
- Subject: CRITICAL: Case Escalation: \${case.name} (3 Weeks Overdue)

8.14 सिस्टम अपडेट {#814-system-updates}

Trigger: Scheduled (daily by default)

Threshold Format: None (system-level)

Worked Example:

Rule Name: "System update notification"

- Recipients: System Administrator, IT Manager
- Subject: OpenEMIS System Update Available: \${new_version}
- Message: A new version \${new_version} is available (current: \${current_version}). Released: \${release_date}.

8.15 कर्मचारी उपस्थिति — कार्यान्वित नहीं {#815-staff-attendance--not-implemented}

Staff Attendance alert type वर्तमान में **implemented नहीं** है। यह Alerts list में दिखता है, लेकिन frequency हमेशा "Never" होता है और नियम तब भी execute नहीं होते हैं।

क्यों implemented नहीं है? Staff attendance tracking अभी तक core OpenEMIS में फीचर नहीं है। यह future release में जोड़ा जाएगा।

यदि आप इसे use करना चाहते हैं: अपने system administrator को बताएं कि आप staff attendance monitoring चाहते हैं।

9. वर्कफ़्लो-ट्रिगर्ड अलर्ट {#9-workflow-triggered-alerts}

9.1 वर्कफ़्लो अलर्ट नियम-आधारित अलर्ट से कैसे भिन्न हैं {#91-how-workflow-alerts-differ-from-rule-based-alerts}

Rule-based alerts:

- Configuration-driven (नियमों के माध्यम से)।
- Multiple rules एक ही alert type के लिए।
- Threshold JSON control करता है जब trigger करें।

Workflow-triggered alerts:

- Step transition पर automatically fire करें (नियमों के बिना)।
- CaseEscalation के साथ integrated (एक case एक step से दूसरे में जाता है)।
- Recipients = assignee + configured roles।
- **Cannot be disabled** — step transition होता है, alert भेजा जाता है (कुछ suppress conditions के साथ)।

9.2 वर्कफ़्लो अलर्ट को दबाने वाली शर्तें {#92-conditions-that-suppress-a-workflow-alert}

यह ईवेंट्स workflow step assignment को trigger नहीं करता है:

शर्त	परिणाम
Step is "Open" state	New cases start in "Open"; no transition alert।
No assignee on the case	No email sent (assignee नहीं है)।
Assignee has no preferred email	No email sent (email address नहीं है)।
Recipient role has no users	No email sent (recipients नहीं हैं)।
Case is already in the step	Only on first entry to the step; no repeat alerts।

9.3 वर्कफ़्लो अलर्ट सेट अप करना {#93-setting-up-a-workflow-alert}

Workflow-triggered alerts को **Workflows > Steps** में configure किया जाता है:

1. **Workflows** पर जाएं।
2. आपकी workflow को edit करें।
3. एक step को edit करें।
4. **Alert on Step Transition** checkbox को check करें।
5. **Notify Roles** से roles select करें।
6. **Alert Subject** और **Alert Message** दर्ज करें।
7. **Save** क्लिक करें।

अब जब कोई case इस step में enter करेगा, assignee और configured roles को automatically alert मिलेगा।

10. अलर्ट क्यू — डिलीवरी पाइपलाइन {#10-alert-queue--delivery-pipeline}

10.1 क्यू स्क्रीन का उद्देश्य {#101-purpose-of-the-queue-screen}

Alert Queue screen सभी **pending** और **failed** messages को दिखाता है। यह debug करने और delivery problems को resolve करने में मदद करता है।

Queue में items वे हैं जो:

- अभी भेजे नहीं गए हैं (status = pending)।
- भेजने का प्रयास विफल हुआ (status = failed)।

Successfully sent messages alert_logs में जाते हैं (अगला section देखें)।

10.2 क्यू कॉलम {#102-queue-columns}

कॉलम	अर्थ
ID	Unique queue item identifier।
Feature	Alert type (e.g., StudentAbsence, CaseEscalation)।
Recipient	Email address या phone number जहाँ message भेजा जाएगा।
Method	Email या SMS।
Status	pending, sent, या failed।
Subject	Email subject (SMS के लिए NA)।
Message	Message body (truncated view; full text click करके देखें)।
Created	जब यह queue item create हुआ।
Sent At	जब message सफलतापूर्वक भेजा गया (या NULL अगर pending/failed)।
Error	Failure reason (अगर status = failed)।

10.3 स्थिति कोड {#103-status-codes}

स्थिति	अर्थ	अगला कदम
pending	Message queue में है, अभी नहीं भेजा गया।	alerts:process command run करें या next cron execution की प्रतीक्षा करें।
sent	Message successfully delivered।	कोई कार्य नहीं आवश्यक (item alert_logs में copy हो गया)।
failed	Delivery attempt विफल (e.g., SMTP error, Twilio error)।	Error message को देखें, issue fix करें, फिर item को retry करें या manually delete करें।

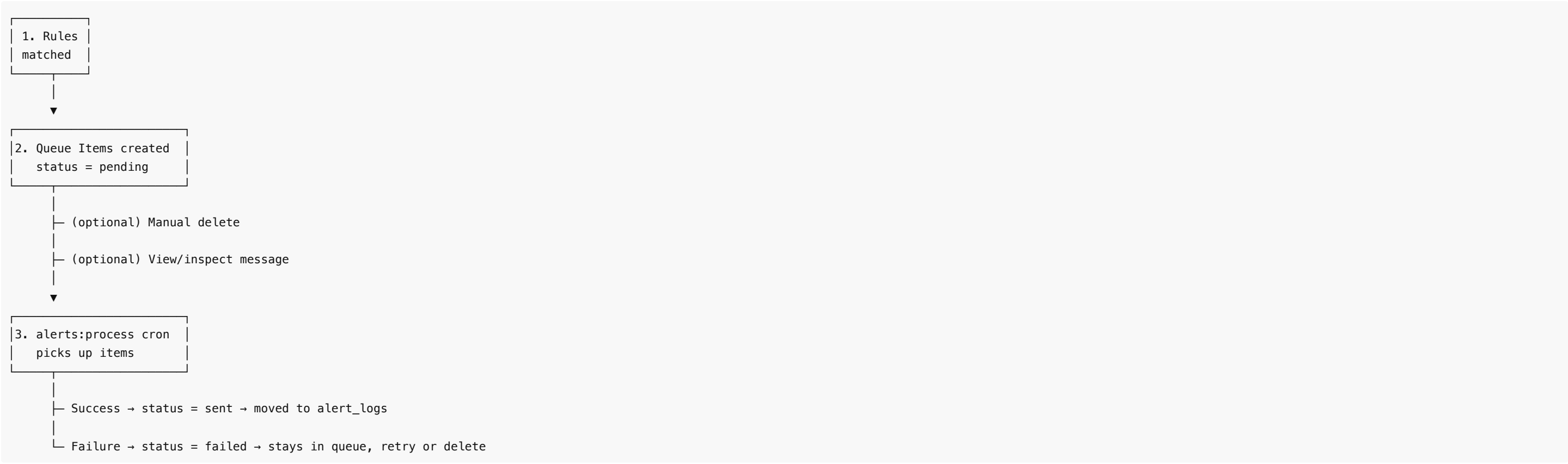
10.4 क्यू आइटम को बड़े पैमाने पर हटाना {#104-mass-deleting-queue-items}

100+ items को एक बार में हटाने के लिए:

- Alert Queue** screen पर जाएं।
- Filter** (funnel icon) पर क्लिक करें।
- Status = "sent" या "failed" select करें (या छोड़ें सभी के लिए)।
- Apply Filter** क्लिक करें।
- Select All** checkbox पर क्लिक करें (all filtered rows चुनने के लिए)।
- Delete Selected** बटन क्लिक करें।
- Confirmation dialog में **Yes** क्लिक करें।

Warning: यह operation irreversible है। आप हटाई गई entries को recover नहीं कर सकते।

10.5 क्यू जीवनचक्र आरेख {#105-queue-lifecycle-diagram}



11. अलर्ट लॉग — ऑडिट ट्रेल {#11-alert-logs--audit-trail}

11.1 अलर्ट लॉग का उद्देश्य {#111-purpose-of-alert-logs}

Alert Logs सभी **successfully sent** messages का permanent audit trail है। Rules, recipients, content, timestamps — सब कुछ।

यह के लिए उपयोगी है:

- Compliance audits (जो notification भेजे गए, कब, किसे) ।
- Duplicate detection (SHA-256 checksums का उपयोग) ।
- Analysis (कौन सी alerts ज्यादा frequently trigger होती हैं) ।

11.2 SHA-256 चेकसम के माध्यम से विस्थापन {#112-deduplication-via-sha-256-checksums}

एक ही message को एक ही recipient को दो बार भेजने से रोकने के लिए, system SHA-256 checksum का उपयोग करता है।

checksum कैसे compute होता है:

```
checksum = SHA-256(subject + "|" + message_body + "|" + recipient_email)
```

अगर एक alert फिर से trigger होता है और same checksum के साथ एक entry `alert_logs` में पहले से exist करता है:

व्यवहार	परिणाम
Within 24 hours	Duplicate माना जाता है। Queue item create नहीं होता।
After 24 hours	New entry allowed (24-hour deduplication window) ।

यह के लिए useful है जहाँ rules multiple times trigger हो सकते हैं (e.g., attendance check daily); आप एक ही notification spam से बचना चाहते हैं।

11.3 एकल लॉग प्रविष्टियों को देखना और हटाना {#113-viewing-and-deleting-single-log-entries}

Single entry देखने के लिए:

1. **Alert Logs** screen पर जाएं।
2. किसी भी log row पर क्लिक करें।
3. Full message content, checksum, timestamp देखें।
4. **Close** बटन दबाएं।

Single entry हटाने के लिए:

1. **Alert Logs** screen पर जाएं।
2. row में **Delete** (trash) icon क्लिक करें।
3. Confirmation में **Yes** क्लिक करें।

11.4 लॉग को बड़े पैमाने पर हटाना {#114-mass-deleting-log-entries}

100+ entries को एक बार में हटाने के लिए:

1. **Alert Logs** screen पर जाएं।
2. **Filter** पर क्लिक करें।
3. Date range select करें (e.g., "Before 2024-01-01")।
4. **Apply Filter** क्लिक करें।
5. **Select All** checkbox दबाएं।
6. **Delete Selected** बटन क्लिक करें।
7. Confirmation में **Yes** क्लिक करें।

Warning: Deleted logs recover नहीं हो सकते। Before mass delete, आप अपने compliance requirements को समझना सुनिश्चित करें।

12. मेसेजिंग (संस्था-स्तर) {#12-messaging-institution-level}

12.1 संस्था-विशिष्ट Email और SMS सेटिंग्स {#121-institution-specific-email-and-sms-settings}

प्रत्येक संस्था अपनी email (SMTP) और SMS (Twilio) सेटिंग्स configure कर सकता है।

यहाँ configure करें:

Institutions > [संस्था नाम] > Communications > Email Settings / SMS Settings

सेटिंग	अर्थ
SMTP Host	Email server (e.g., mail.example.com)।
SMTP Port	Port number (e.g., 587, 465)।
SMTP Username	Authentication user।
SMTP Password	Authentication password।
Twilio Account SID	SMS provider account ID।
Twilio Auth Token	SMS provider token।
Twilio Phone Number	SMS sender number।

12.2 Default Messaging विकल्प {#122-default-messaging-options}

अगर कोई संस्था settings configure नहीं करता:

Method	Default
--------	---------

Email	System-level SMTP settings use होता है (यदि configured)।
SMS	SMS भेजे नहीं जाते (provider configured नहीं है)।

12.3 ईमेल और SMS कॉन्टैक्ट सत्यापन {#123-email-and-sms-contact-verification}

Alerts deliver होने से पहले, सिस्टम सत्यापित करता है कि recipients के पास valid contact है:

Method	Validation
Email	User record में email field filled और valid format।
SMS	User record में phone field filled और valid format।

अगर recipients के पास invalid/missing contact हैं, alert delivery skip होता है (error logged)।

13. परिचालन कॉन्फ़िगरेशन {#13-operational-configuration}

13.1 ALERTS_PROCESS_LIMIT और थ्रॉटलिंग {#131-alerts_process_limit-and-throttling}

Large institutions पर, एक ही cron execution में 1000+ alerts send हो सकते हैं। Email/SMS provider rate limits के कारण, आप throttle कर सकते हैं:

.env file में:

```
ALERTS_PROCESS_LIMIT=100
```

मतलब: `alerts:process` cron एक बार में maximum 100 messages भेजता है, फिर पुनः restart होता है।

Setting recommendations:

Institution Size	Recommended Limit
Small (< 500 students)	50–100 messages/run
Medium (500–5000 students)	100–200 messages/run
Large (5000+ students)	200–500 messages/run

Note: छोटा limit = ज्यादा frequent cron runs, लेकिन slow message delivery।

13.2 System Process शेड्यूल {#132-system-process-schedule}

Alert commands को निम्नानुसार schedule किया जाना चाहिए:

कमांड	आवृत्ति	कॉन्फ़िगरेशन
<code>alerts:process</code>	Every 5 minutes	<code>* /5 * * * * cd /var/www/html/emis/core/api && php artisan alerts:process</code>
<code>alerts:check-and-queue</code>	Every 24 hours	<code>0 2 * * * cd /var/www/html/emis/core/api && php artisan alerts:check-and-queue (2 AM)</code>

13.3 अलर्ट लॉग को साफ़ करना {#133-cleaning-alert-logs}

Alert logs exponentially grow करते हैं। पुराने entries को clean करने के लिए:

Manual cleanup:

```
cd /var/www/html/emis/core/api
php artisan alerts:clean-logs --days=90
```


(सभी logs 90 दिनों से पहले delete करें)

Automated cleanup (cron):

```
0 3 * * 0 cd /var/www/html/emis/core/api && php artisan alerts:clean-logs --days=90
```

(हर Sunday को 3 AM पर)

14. परीक्षण और सूखा चलाना प्रक्रिया {#14-testing-dry-run-procedures}

14.1 अलर्ट नियम सूखा चलाना {#141-dry-running-an-alert-rule}

एक rule को **test करने के लिए** production data को बिना affect किए:

- 1. Alert Rules screen पर, rule को edit करें।
- 2. **Test Rule** बटन पर क्लिक करें (यदि available)।
- 3. सिस्टम matching records को query करेगा और preview दिखाएगा, लेकिन queue में कुछ नहीं डालेगा।
- 4. Output देखें (कितने records match, किन recipients को notify किया जाएगा)।

14.2 व्यक्तिगत अलर्ट देखना {#142-viewing-individual-alerts}

एक specific alert record को manually inspect करने के लिए:

```
cd /var/www/html/emis/core/api
php artisan alerts:inspect --feature=StudentAbsence --record_id=12345
```

यह command दिखाएगा:

- जो data resolve होगा।
- कौन से rules match होते हैं।
- Preview of messages।

14.3 डेटाबेस अनुमति यांत्रिकी को समझना {#143-understanding-database-anonymisation-mechanics}

Alerts production से अलग test database पर काम करते हैं जहाँ sensitive student/staff data anonymized हो सकता है। यह सुनिश्चित करता है कि test alerts production customers को नहीं जाते।

14.4 डेटाबेस अनुमति यांत्रिकी को समझना {#144-verifying-database-is-anonymised}

एक test database anonymized है verify करने के लिए:

```
cd /var/www/html/emis/core/api

# Real institution data count
mysql -h 127.0.0.1 -u root -prootpassword openemis_core_v5  "SELECT COUNT(*) FROM institutions WHERE code LIKE '%GOV%';"

# Anonymised database should return 0 or low numbers
mysql -h 127.0.0.1 -u root -prootpassword openemis_core_test  "SELECT COUNT(*) FROM institutions WHERE code LIKE '%GOV%';"
```

14.5 सभी निर्धारित अलर्ट को तुरंत चलाने को बाध्य करना {#145-force-running-all-scheduled-alerts-now}

Normally, scheduled alerts daily cron से चलते हैं। तुरंत force करने के लिए (testing के लिए):

```
cd /var/www/html/emis/core/api
php artisan alerts:check-and-queue --force --sync
```

Option	मतलब
--------	------

--force	Frequency schedule को ignore करें (e.g., "Never" भी run करें)।
--sync	Don't background the job; wait for completion.

15. समस्या निवारण {#15-troubleshooting}

15.1 अलर्ट फायर किया गया लेकिन कोई ईमेल नहीं मिला {#151-alert-fired-but-no-email-received}

Diagnosis:

- Alert Logs में entry है (message sent)?
 - हाँ → Email provider issue (SMTP failure, spam folder) । Email server logs check करें।
 - नहीं → Alert rule match नहीं हुआ, या threshold शर्त पूरी नहीं हुई।
- Alert Queue में entry है?
 - हाँ, status = "pending" → `alerts:process` cron नहीं चल रहा है। Cron schedule verify करें।
 - हाँ, status = "failed" → Error column में failure reason देखें।

Fix Steps:

```
# Check if cron is running
ps aux | grep "php artisan alerts:process"

# Run manually
cd /var/www/html/emis/core/api
php artisan alerts:process

# Check logs
tail -f /var/www/html/emis/core/logs/alert_process.log
```

15.2 अलर्ट नियम सक्षम लेकिन कभी फायर नहीं होता {#152-alert-rule-enabled-but-never-fires}

Diagnosis:

- Alert type frequency = "Never"?
 - Alerts screen पर alert को edit करें, frequency को enable करें।
- Threshold मान सही हैं?
 - `${value}` = 30 मतलब "30 दिन से ज्यादा"। आपका data 30+ दिन condition को पूरा करता है?
 - Test rule dry-run option के साथ।
- कोई matching records हैं?
 - SQL में manually query करें:

```
SELECT * FROM students
WHERE DATEDIFF(CURDATE(), last_attendance_date) >= 30;
```

Fix Steps:

```
# Test the rule manually
cd /var/www/html/emis/core/api
php artisan alerts:inspect --feature=StudentAbsence --rule_id=42
```


15.3 वर्कफ़्लो अलर्ट फायर नहीं हो रहा {#153-workflow-alert-not-firing}

Diagnosis:

- 1. Step में workflow alert configured है?
 - Workflows > [Workflow] > Steps > [Step] check करें, "Alert on Step Transition" enabled है?
- 2. Assignee exists है?
 - Case को edit करें, **Assigned To** field भरा हुआ है?
- 3. Assignee के पास preferred email है?
 - Security Users को check करें, assignee को email address set है?

Fix Steps:

```
-- Check if step has alert configured
SELECT * FROM workflow_steps WHERE id = 5;

-- Check case assignment
SELECT assigned_to_user_id, assigned_to_role_id FROM cases WHERE id = 123;

-- Check user email
SELECT id, email FROM security_users WHERE id = 456;
```

15.4 क्यू में डुप्लिकेट अलर्ट {#154-duplicate-alerts-in-queue}

Diagnosis:

- 1. Message subject/body exactly same हैं?
 - हाँ → Deduplication logic same checksum detect करता है (24-hour window) ।
 - नहीं → Different messages, expected behavior ।
- 2. Recipients same हैं?
 - हाँ → Multiple rules matching (expected) ।

Fix Steps:

```
# Check checksum for recent alerts
SELECT id, feature, checksum, created FROM alert_logs
WHERE created > DATE_SUB(NOW(), INTERVAL 24 HOUR)
GROUP BY checksum
HAVING COUNT(*) > 1;
```

यदि आप duplicates को disable करना चाहते हैं, आप alert_logs_deduplication_enabled config को false set कर सकते हैं (administrator की आवश्यकता है)।

15.5 बड़े पैमाने पर हटाना सभी चयनित पंक्तियों को हटाता नहीं है {#155-mass-delete-does-not-remove-all-selected-rows}

Diagnosis:

- 1. Pagination: सभी चयनित rows एक ही page पर हैं?
 - नहीं → Select All केवल current page को select करता है, सभी को नहीं।
- 2. Large delete: 10000+ rows?
 - Database timeout हो सकता है।

Fix Steps:

- 1. Filter करें ताकि remaining rows < 500।
- 2. फिर से Select All करें।
- 3. Delete करें।
- 4. दोहराएँ।

या SQL से directly delete करें:

```
cd /var/www/html/emis/core/api
mysql -h 127.0.0.1 -u root -prootpassword openemis_core_v5  "DELETE FROM alert_queue WHERE status = 'sent' AND created < DATE_SUB(NOW(), INTERVAL 30 DAY) LIMIT 5000;"
```

15.6 क्यू बैकिंग अप (संदेश भेज नहीं रहे) {#156-queue-backing-up-messages-not-sending}

Diagnosis:

- 1. alerts:process cron running है?

```
ps aux | grep "php artisan alerts:process"
```

- 2. SMTP server down है?

```
telnet mail.example.com 587
```

- 3. Queue में कितने pending items हैं?

```
SELECT COUNT(*) FROM alert_queue WHERE status = 'pending';
```

Fix Steps:

```
# Manually run process with increased limit
cd /var/www/html/emis/core/api
php artisan alerts:process --force --limit=500

# Check logs
tail -100 /var/www/html/emis/core/logs/alert_process.log

# If SMTP error, test SMTP
telnet mail.example.com 587
```

15.7 कमांड लॉग फ़ाइलों को पढ़ना {#157-reading-the-command-log-files}

Alert command logs यहाँ स्थित हैं:

```
/var/www/html/emis/core/logs/alert_*.log      # Alert-specific logs
/var/www/html/emis/core/logs/system_processes/  # Per-command execution logs
```

Most recent log देखने के लिए:

```
tail -50 /var/www/html/emis/core/logs/alert_student_absence.log
tail -50 /var/www/html/emis/core/logs/alert_process.log
```

Specific command execution log:

```
ls -ltr /var/www/html/emis/core/logs/system_processes/ | tail -5
cat /var/www/html/emis/core/logs/system_processes/12345.log
```

Log format है:

```
[2026-04-15 10:30:45] INFO: alerts:check-and-queue starting
[2026-04-15 10:30:46] INFO: Found 3 rules for StudentAbsence
[2026-04-15 10:30:47] INFO: Queued 12 messages for delivery
[2026-04-15 10:30:48] INFO: alerts:check-and-queue completed
```

16. परिशिष्ट {#16-appendices}

A. पूर्ण Artisan कमांड संदर्भ {#a-full-artisan-command-reference}

कमांड	Feature	Trigger	विकल्प
alerts:student-absence	StudentAbsence	Event	--user_id, --rule_id, --process_id
alerts:student-admission	StudentAdmission	Event	--user_id, --rule_id, --process_id
alerts:student-enrolment	StudentEnrolment	Event	--user_id, --rule_id, --process_id
alerts:student-status	StudentStatus	Event	--user_id, --rule_id, --process_id
alerts:retirement-warning	RetirementWarning	Scheduled	--user_id, --rule_id, --process_id
alerts:staff-employment	StaffEmployment	Event	--user_id, --rule_id, --process_id
alerts:staff-leave	StaffLeave	Event	--user_id, --rule_id, --process_id
alerts:staff-type	StaffType	Scheduled	--user_id, --rule_id, --process_id
alerts:license-validity	LicenseValidity	Scheduled	--user_id, --rule_id, --process_id
alerts:license-renewal	LicenseRenewal	Scheduled	--user_id, --rule_id, --process_id
alerts:scholarship-application	ScholarshipApplication	Scheduled	--user_id, --rule_id, --process_id
alerts:scholarship-disbursement	ScholarshipDisbursement	Scheduled	--user_id, --rule_id, --process_id
alerts:case-escalation	CaseEscalation	Scheduled	--user_id, --rule_id, --process_id
alerts:system-updates	SystemUpdates	Scheduled	--user_id, --rule_id, --process_id

Note: *StaffAttendance* (*alerts:staff-attendance*) exist नहीं करता है और सूचीबद्ध नहीं है — §8.15 देखें।

B. तीन कमांड-मैप चेकलिस्ट {#b-three-command-maps-checklist}

जब system में एक नई alert type जोड़ते हैं, तीन source locations को update किया जाना चाहिए। किसी एक को omit करने से नई alert silently fail होगी या सही dispatch path में नहीं दिखेगी।

Map 1 — Event-based alerts (CakePHP side): File: `plugins/Alert/src/Model/Table/AlertLogsTable.php` Method: `triggerAlertCommand()` Purpose: `process_name` को artisan command string में map करता है alerts को dispatch करने के लिए जो CakePHP `afterSave()` events से trigger होते हैं।

Map 2 — Scheduled alerts (Laravel side): File: `api/app/Console/Commands/CheckAndQueueAlerts.php` Method: `queueAlertCommand()` Purpose: `process_name` को artisan command string में map करता है scheduled alerts के लिए केवल। Event-based commands को इस map में comment out किया जाना चाहिए double-dispatch से बचने के लिए।

Map 3 — Event-based alerts (Laravel side): File: `api/app/Services/AlertTriggerService.php` Method: `triggerAlertCommand()` Purpose: Laravel side से trigger होने वाली event-based commands के लिए process names को map करता है। केवल event-based commands को इस map में रखना चाहिए।

C. SQL संदर्भ क्वेरी {#c-sql-reference-queries}

```
-- CaseEscalation threshold में use करने के लिए workflow step IDs खोजें
SELECT ws.id, ws.name, w.name AS workflow
FROM workflow_steps ws
JOIN workflows w ON w.id = ws.workflow_id
ORDER BY w.name, ws.name;

-- StaffLeave threshold में use करने के लिए staff leave types सूचीबद्ध करें
SELECT id, name FROM staff_leave_types ORDER BY name;

-- LicenseRenewal threshold में use करने के लिए staff training categories सूचीबद्ध करें
SELECT id, name FROM staff_training_categories ORDER BY name;

-- StaffType threshold में use करने के लिए staff types सूचीबद्ध करें
SELECT id, name FROM staff_types ORDER BY name;

-- LicenseValidity और LicenseRenewal threshold में use करने के लिए license types सूचीबद्ध करें
SELECT id, name FROM license_types ORDER BY name;

-- StudentStatus threshold में use करने के लिए student statuses सूचीबद्ध करें
SELECT id, name FROM student_statuses ORDER BY name;

-- ScholarshipApplication threshold में use करने के लिए workflow step categories सूचीबद्ध करें
SELECT DISTINCT category FROM workflow_steps
WHERE category IS NOT NULL
ORDER BY category;
```

D. शब्दावली {#d-glossary}

Term	परिभाषा
Feature Key	Exact string identifier एक alert type के लिए जो alert_rules.feature column में दिखता है। Case-sensitive। उदाहरण: StudentAttendance, CaseEscalation।
Process Name	PHP class name internally एक artisan command को identify करने और इसे dispatch path में map करने के लिए। उदाहरण: AlertStudentAbsence। Feature key से अलग।
Checksum	SHA-256 hash एक specific recipient के लिए resolved subject और message body का। Duplicate delivery को detect और prevent करने के लिए alert_logs.checksum में use होता है।
Recipient Resolver	Laravel service class (RecipientResolver) जो security_groups, security_group_users, और security_users को query करता है एक specific institution और security roles के लिए एक email addresses और phone numbers की list produce करने के लिए।
Threshold JSON	JSON-formatted value alert_rules.threshold में stored जो control करता है कि कौन से records notification के लिए qualify करते हैं। Format और required fields alert type के आधार पर differ करते हैं।
Queue Item	Single row alert_queue में एक pending, sent, या failed message delivery attempt को represent करता है एक recipient के लिए।
System Process	Row system_processes में एक artisan alert command के एक execution को represent करता है। Command name, status, और execution log file के path को store करता है।

E. आगे पढ़ना {#e-further-reading}

Document	Location	Description
README.md	api/storage/release-docs/POCOR-9509/README.md	POCOR-9509 के लिए release summary, deployment checklist, और getting-started links।

ALERTS_GUIDE.md	api/storage/release-docs/POCOR-9509/ALERTS_GUIDE.md	Developers के लिए technical implementation guide; architecture internals, three command maps, और activation checklist को cover करता है।
thresholds.md	api/storage/release-docs/POCOR-9509/thresholds.md	Complete threshold configuration reference validation rules और multi-rule strategy patterns के साथ।